Project Report: Sales & Order Management Analytics System

1. Project Information Header

- **Participant Name:** Nishtha Garg
- **Course Name:** Data Analyst Training Program
- **Course Duration:** July (3 Months)
- **Mode of Learning:** Online
- **Project Title:** Sales & Order Management Analytics System
- **Database Platform:** MySQL

2. Abstract

The Sales & Order Management Analytics System is a relational SQL project designed for an e-commerce or retail environment. The system manages customer, product, order, order-item, and payment information and uses SQL to generate business-oriented analytics. The implementation demonstrates database creation, data manipulation, joins, aggregations, subqueries, common table expressions (CTEs), CASE expressions, window functions, reusable views, indexes, query-plan analysis, and transaction management.

The project uses MySQL and is structured around normalized relational tables connected through primary and foreign keys. Analytical queries are used to examine revenue, order volume, customer performance, product performance, category performance, payment status, and month-over-month revenue changes.

3. Introduction

Retail and e-commerce businesses generate large amounts of transactional data. A structured relational database makes it possible to store this information consistently and analyze it to support operational and business reporting. This project applies SQL concepts to a practical sales and order management scenario.

The system contains five main entities: customers, products, orders, order_items, and payments. These entities are connected through keys so that customer purchases, products within orders, and associated payments can be analyzed together.

4. Problem Statement

The objective is to build a relational sales and order management database and use SQL analytics to answer practical business questions. The system should support data creation and maintenance while also providing reports such as total revenue, top products, customer rankings, monthly sales, category performance, and payment analysis.

5. Objectives

- Design a normalized relational database for sales and order management.
- Create tables with primary keys, foreign keys, and data constraints.
- Insert realistic sample customer, product, order, order-item, and payment data.
- Demonstrate CRUD operations and different types of SQL joins.
- Use aggregate functions, GROUP BY, and HAVING for business reporting.
- Apply subqueries, CTEs, CASE expressions, and window functions.
- Create reusable SQL views for reporting.
- Use indexes and EXPLAIN to demonstrate query optimization.
- Demonstrate transactions using COMMIT, ROLLBACK, and SAVEPOINT.
- Generate business-oriented sales, customer, product, and payment reports.

6. Technologies Used

Technology / Tool	Purpose
MySQL	Relational database management and SQL execution
MySQL Workbench	Database creation, query execution, and result inspection
SQL	Data definition, manipulation, querying, analytics, and optimization

7. Database Design

The database is named sales_order_analytics. It contains five related tables. Primary keys uniquely identify records, while foreign keys establish relationships between customers, orders, products, order items, and payments.

7.1 Database Tables

Table	Main Columns	Purpose
customers	customer_id, name, email, city, signup_date	Stores customer information
products	product_id, product_name, category, price, stock	Stores product and inventory information
orders	order_id, customer_id, order_date, status, total_amount	Stores customer orders
order_items	order_item_id, order_id, product_id, quantity, unit_price	Stores products contained in orders
payments	payment_id, order_id, payment_date, amount, payment_status	Stores payment transactions

7.2 Relationships

- customers.customer_id → orders.customer_id
- orders.order_id → order_items.order_id
- products.product_id → order_items.product_id
- orders.order_id → payments.order_id

This structure allows the system to connect customers with their orders, orders with their products, and orders with their payments.

8. Database Implementation

8.1 DDL and Constraints

The database and five tables were created using CREATE DATABASE and CREATE TABLE statements. Primary keys were used for unique record identification, while foreign keys were used to maintain relationships between tables. NOT NULL and UNIQUE constraints were also applied where appropriate.

8.2 Data Insertion

Sample data was inserted into all five tables. The dataset contains multiple customers, products, orders, order items, and payment records with different order and payment statuses. This provides enough variation for joins, aggregations, rankings, and analytical reporting.

8.3 CRUD Operations

CRUD operations were demonstrated using INSERT for record creation, SELECT for reading data, UPDATE for modifying a customer record, and DELETE for removing a temporary test record. This demonstrates basic database maintenance operations.

9. SQL Querying and Analysis

9.1 Joins

INNER JOIN was used to combine customers with orders and orders with order items and products. Multi-table joins were also used to combine customer, order, and payment information. LEFT JOIN was used to identify customers with and without orders. RIGHT JOIN and a UNION-based FULL OUTER JOIN simulation were also demonstrated for MySQL.

9.2 Aggregations

SUM, COUNT, AVG, MIN, and MAX were used to calculate total revenue, order counts, average order value, minimum order value, and maximum order value. GROUP BY and HAVING were used for monthly, product, category, and customer-level analysis.

9.3 Subqueries and CTEs

Subqueries were used to identify orders above average order value, customers spending above the average customer revenue, and the most expensive product. CTEs were used to create intermediate customer-revenue and monthly-sales datasets for clearer analytical queries.

9.4 CASE Expressions

CASE expressions were used to classify customers into revenue segments, orders into value categories, and payments into meaningful status categories.

9.5 Window Functions

RANK and DENSE_RANK were used for customer and product rankings. ROW_NUMBER was used to number customer orders. SUM with an OVER clause was used for running revenue, while LAG was used to compare current and previous order or monthly revenue values.

10. Business Analytics Reports

- **10.1 Overall Sales Summary:** Calculates completed order count, total revenue, average order value, minimum order value, and maximum order value. These measures provide a high-level summary of completed sales activity.
- **10.2 Monthly Sales Analysis:** Completed orders are grouped by year and month to calculate monthly order volume, monthly revenue, and average order value. This provides a basis for identifying changes in sales activity over time.
- **10.3 Product Performance:** Products are analyzed using units sold and revenue generated from completed orders. The report can be sorted by revenue to identify the highest-revenue products.
- **10.4 Customer Performance:** Customer-level reporting calculates total orders, total revenue, and average order value. A RANK window function is used to create a revenue-based customer ranking.
- **10.5 Category Performance:** Product categories are compared using units sold and category revenue. This allows the sales mix to be examined at category level.
- **10.6 Payment Analysis:** Payment records are grouped by payment status to show the number and total amount of payments associated with each status, including PAID, PENDING, and REFUNDED records.
- **10.7 Month-over-Month Analysis:** A CTE and LAG window function are used to compare each month's revenue with the previous month. The query calculates both absolute revenue change and percentage month-over-month growth.

11. SQL Views

Four reusable views were created to simplify recurring reporting tasks:

- vw_customer_revenue: Customer order count, revenue, and average order value.
- vw_product_performance: Product units sold and revenue.
- vw_monthly_sales: Monthly orders, revenue, and average order value.
- vw_order_payment_details: Combined customer, order, and payment information.

12. Indexing and Query Optimization

Indexes were created on frequently used analytical columns such as order date, order status, order-item product ID, and payment status. A composite index on order status and order date was also demonstrated for queries filtering on both attributes.

EXPLAIN was used to inspect the MySQL query execution plan. The execution plan can be used to examine possible indexes, the selected index, estimated rows examined, and other execution details. This provides a practical demonstration of query optimization.

13. Transaction Management

Transaction control was demonstrated using START TRANSACTION, SAVEPOINT, ROLLBACK TO, ROLLBACK, and COMMIT. Temporary test customer records were used so that transaction behavior could be demonstrated without altering the core project dataset.

The transaction workflow demonstrated that changes can be temporarily staged, partially reversed to a savepoint, completely rolled back, or permanently committed.

14. Key Business Insights Framework

The implemented queries support the following business questions:

- How much revenue was generated from completed orders?
- How many completed orders were placed?
- What is the average order value?
- Which products generated the highest revenue?
- Which customers generated the highest completed-order revenue?
- Which product categories generated the most revenue?
- What are the distributions of PAID, PENDING, and REFUNDED payments?
- How does monthly revenue change from one month to the next?
- How can recurring reports be simplified using SQL views?
- How can indexes and EXPLAIN help analyze query performance?

Note: Numerical business findings should be taken directly from the final MySQL query outputs. This report intentionally does not invent or assume numerical results that are not shown in the project evidence.

15. Conclusion

The Sales & Order Management Analytics System demonstrates the use of MySQL for both relational data management and business analytics. The project integrates customers, products, orders, order items, and payments into a connected relational structure and applies a broad range of SQL techniques to analyze the resulting data.

The implementation covers database design, CRUD operations, joins, aggregate functions, subqueries, CTEs, CASE expressions, window functions, views, indexing, EXPLAIN-based optimization, and transactions. Together, these components provide a practical end-to-end example of using SQL for sales and order analytics.

16. Future Scope

- Connect the database to a BI tool such as Power BI or Tableau for interactive dashboards.
- Add stored procedures and triggers for automated business operations.
- Introduce automated ETL pipelines for larger or continuously updated datasets.
- Add advanced data warehousing or historical-data techniques where required.
- Extend the system with additional sales, inventory, and customer-retention metrics.

17. Final Project Deliverables

Deliverable	Description
SQL Script	Database creation, data insertion, CRUD, joins, analytics, views, indexes, optimization, and transactions
Project Report	Documentation of objectives, database design, SQL techniques, analytics, optimization, and conclusion